

Java Backend Architecture

Interview Series

Chapter 14.3

Consistent Hashing & Load Balancing

50+ Interview Q&A

Code Examples

Architecture Diagrams

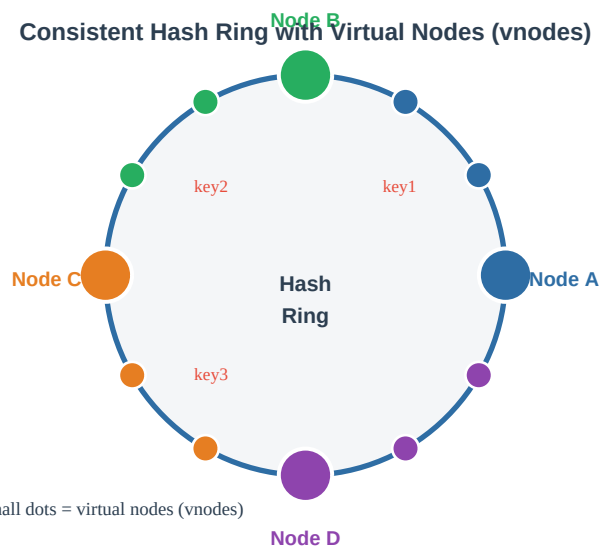
Advanced Topics

Chapter 14.3 – Consistent Hashing & Load Balancing

Overview

Consistent hashing is the foundation of distributed caching and data sharding, minimizing rehashing when nodes change. Load balancing distributes traffic across healthy instances using algorithms tailored to different workload characteristics. Together they enable resilient, horizontally scalable distributed systems.

Consistent Hashing



- Hash ring: 0 to $2^{32}-1$ values arranged in a circle
- Each node maps to one or more positions on the ring via $\text{hash}(\text{node_id})$
- Each key maps to the nearest clockwise node: $\text{hash}(\text{key}) \rightarrow$ walk clockwise
- Adding a node: only keys between the new node and its predecessor migrate
- Removing a node: only that node's keys migrate to its successor
- With N nodes: adding/removing redistributes K/N keys (K = total keys)
- Without consistent hashing (modulo): adding 1 node to N $\rightarrow$ almost all K keys remapped

Virtual Nodes (Vnodes)

- Problem: with few physical nodes, keys distribute unevenly (hot spots)
- Solution: each physical node maps to multiple positions on the ring (vnodes)
- Example: 4 physical nodes $\times$ 150 vnodes each = 600 ring positions $\rightarrow$ uniform distribution
- Cassandra default: 256 vnodes per node. DynamoDB: uses vnodes internally
- Benefit: when a node is added, it takes vnodes from ALL existing nodes $\rightarrow$ balanced load
- Downside: more memory for node metadata. Trade-off worth it at scale

Rendezvous Hashing

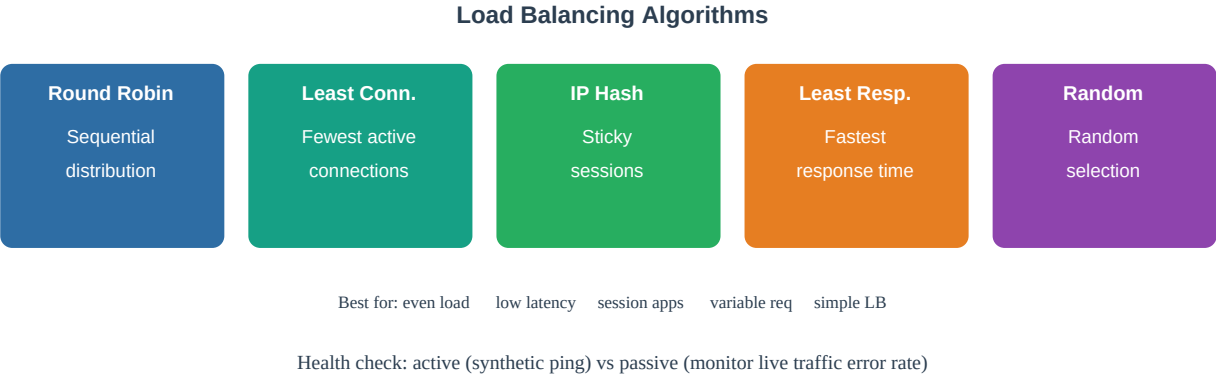
- Alternative to ring: for each key, compute score = $\text{hash}(\text{key}, \text{node})$ for ALL nodes
- Assign key to node with highest score (Highest Random Weight algorithm)

- Advantage: simpler implementation, no ring data structure needed
- Disadvantage: O(N) computation per lookup (vs O(log N) for ring with binary search)
- Used in: CDN cache servers, anycast routing, some load balancers

Java: Consistent Hash Ring Implementation

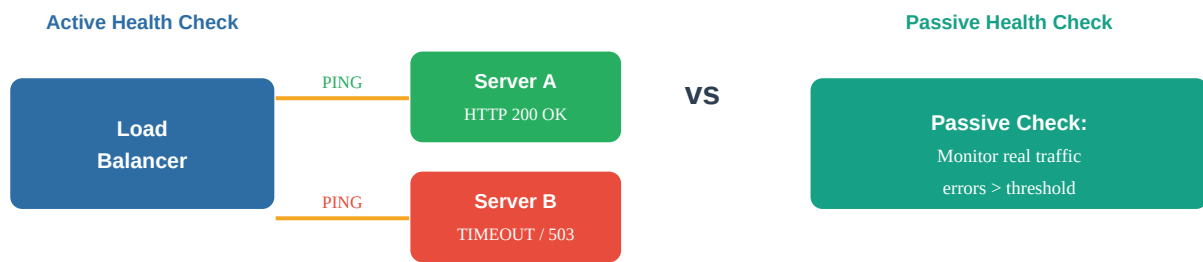
```
import java.util.TreeMap; import java.security.MessageDigest; public class ConsistentHashRing { private final TreeMap<Long, String> ring = new TreeMap<>(); private final int vnodes; public ConsistentHashRing(int vnodes) { this.vnodes = vnodes; } public void addNode(String node) { for (int i = 0; i < vnodes; i++) { ring.put(hash(node + "-vn" + i), node); } } public void removeNode(String node) { for (int i = 0; i < vnodes; i++) { ring.remove(hash(node + "-vn" + i)); } } public String getNode(String key) { if (ring.isEmpty()) return null; Long h = hash(key); // Nearest clockwise node Map.Entry<Long, String> e = ring.ceilingEntry(h); return (e != null ? e : ring.firstEntry()).getValue(); } private long hash(String key) { // MD5 truncated to long for ring position try { byte[] d = MessageDigest.getInstance("MD5").digest(key.getBytes()); return ((long)(d[3]&0xFF)<<24)|((long)(d[2]&0xFF)<<16)|((long)(d[1]&0xFF)<<8)|(d[0]&0xFF); } catch (Exception e) { throw new RuntimeException(e); } } }
```

Load Balancing Algorithms



Algorithm	How It Works	Best For	Avoid When
Round Robin	Each request → next server in sequence	Homogeneous servers, simple LB	Servers with different capacity
Weighted Round Robin	More powerful servers get more requests proportionally	Mixed server capacities	Dynamic server performance changes
Least Connections	Route to server with fewest active connections	Variable request durations	WebSockets, long requests (overhead > benefit)
IP Hash	hash(client_IP) → same server always	Sticky sessions, shopping carts	Clients behind NAT (many to same IP)
Least Response Time	Track response time + active connections, route to fastest	Latency sensitive apps	High overhead tracking metrics
Random	Random server selection	Simple even distribution, testing	Requiring session affinity or optimal routing

Health Checks: Active vs Passive

**Active (synthetic probes) vs Passive (real traffic monitoring) Health Checks**

Active Health Check	Load balancer sends periodic synthetic requests to each server (e.g., GET /health every 5s). Mark server unhealthy if response is non-2xx or times out. Fast detection: 2 failed checks in 10s → remove from pool. Overhead: generates synthetic traffic.
Passive Health Check	LB monitors real request traffic. If error rate from a server exceeds threshold (e.g., >50% 5xx in 30s) → remove from pool. No synthetic traffic, but detection is slower (must wait for real failures). NGINX upstream: max_fails + fail_timeout.
Health Check Endpoint	/actuator/health (Spring Boot) returns JSON with component status. Return 200 OK when ready to serve traffic, 503 when not (during startup, graceful shutdown). Kubernetes: readinessProbe and livenessProbe use different endpoints.
Graceful Drain	Before removing from LB, server: stops accepting new connections (readiness → FAIL), completes in-flight requests (drain timeout: 30s), then shuts down. ALB connection draining: sends FIN after deregister, allows up to 300s for in-flight to complete.

Interview Q&A; — Consistent Hashing & Load Balancing

Q1. What is consistent hashing and why is it used?

Consistent hashing is a distributed hashing scheme where both keys and servers are placed on a hash ring (0 to $2^{32}-1$). Each key routes to the nearest clockwise server. When servers are added or removed, only K/N keys need remapping (K = keys, N = nodes), versus nearly all keys with simple modulo hashing. Used in: distributed caches (Memcached, Redis Cluster), DHTs, CDN routing, Cassandra partitioning.

Q2. How many keys are remapped when a node is added in consistent hashing?

With N nodes and K total keys uniformly distributed: adding 1 node remaps approximately $K/(N+1)$ keys — only the keys that fell between the new node and its predecessor on the ring. Example: 4 nodes, 1M keys → adding 1 node remaps ~200K keys (20%). Simple modulo hashing ($\text{key} \% N$): adding 1 node remaps ~833K keys (83%). Consistent hashing is vastly better.

Q3. What problem do virtual nodes solve in consistent hashing?

With few physical nodes, hash function may distribute ring positions unevenly, causing some nodes to handle much more data (hot spots). Virtual nodes (vnodes) assign each physical node multiple positions on the ring. With 150 vnodes per node, positions are statistically uniform. Added benefit: when adding a node, it takes a few vnodes from every existing node, distributing the migration load evenly.

Q4. How does Cassandra use consistent hashing?

Cassandra assigns each node one or more token ranges on a ring. With vnodes (default: 256 per node), each node owns 256 non-contiguous token ranges. Partitioner (Murmur3Partitioner) hashes partition key → token → nearest node. Replication factor N means N consecutive nodes on the ring store copies. Adding a node: token ranges automatically rebalanced across cluster.

Q5. What is rendezvous hashing?

For each key, compute a score for every node: $\text{score}(\text{key}, \text{node}) = \text{hash}(\text{key} + \text{node_id})$. Assign key to the node with the highest score. Advantage: simpler implementation, no ring data structure. Naturally handles node addition/removal — only keys whose highest-score node changed are remapped. Disadvantage: $O(N)$ per lookup. Used in CDNs for cache server selection.

Q6. What is the difference between round robin and least connections load balancing?

Round robin: distributes requests sequentially regardless of server state. Simple and efficient for homogeneous short requests. Least connections: routes to the server with fewest active concurrent connections. Better for variable request duration (WebSocket, long polling, streaming). Round robin can overload a slow server with many in-flight requests; least connections adapts to actual server load.

Q7. When would you use IP hash load balancing?

IP hash computes $\text{hash}(\text{client_IP}) \rightarrow$ deterministic server. Same client always routes to same server — "sticky sessions". Use when: application has in-process session state (legacy apps that haven't been made stateless), WebSocket connections must maintain server affinity, server-side caching per-user improves performance. Avoid: clients behind shared NAT (many users from one IP → hot server), NAT changes IP mid-session.

Q8. What is the difference between active and passive health checks?

Active: load balancer sends periodic synthetic probes (GET /health every 5-10s). Fast detection, predictable behavior, but adds synthetic traffic. Passive: LB monitors real request errors — if a server returns 5xx errors above threshold, mark unhealthy. No extra traffic, but detection requires real failures. NGINX supports both. AWS ALB: active only. Combine both for best coverage: active detects server-level failures, passive detects application-level errors.

Q9. What should the /health endpoint return?

Return HTTP 200 with JSON body detailing component status when healthy. Return 503 with details when unhealthy or not ready. Spring Boot Actuator: /actuator/health returns {"status":"UP"} with DB, Redis, Kafka sub-checks. Kubernetes splits into: /readiness (ready to serve traffic — DB connected, cache warm) and /liveness (JVM alive — should not restart). Never include sensitive info in health endpoint.

Q10. What is the difference between layer 4 and layer 7 load balancing?

Layer 4 (transport): routes based on TCP/UDP IP address and port. Cannot inspect request content. Fast, low overhead. Used for: non-HTTP protocols, extreme throughput requirements. Layer 7 (application): inspects HTTP headers, URL, cookies. Enables: path-based routing (/api → service A, /static → CDN), host-based routing, SSL termination, request manipulation, WebSocket support. AWS ALB is L7; NLB is L4.

Q11. How does weighted round robin work?

Assigns a weight proportional to each server's capacity. Server A (weight=3): handles 3 requests per cycle. Server B (weight=1): handles 1 request per cycle. Useful when servers have different specs (some have more CPU/RAM). Example: 4-core server gets weight=4, 2-core gets weight=2. Implemented in NGINX (weight parameter in upstream block) and Haproxy (weight directive).

Q12. What happens to in-flight requests when a server is removed from LB pool?

Without graceful drain: LB immediately stops routing new requests to server, but existing connections are dropped mid-request → client errors. With connection draining: LB marks server as "draining", stops new requests, allows existing connections up to drain timeout (30-300s) to complete. AWS ALB deregistration delay: 300s default (tunable to 0-3600s). Essential for zero-downtime deployments.

Q13. What is the C10K problem and how does it relate to load balancing?

C10K: handling 10,000 concurrent client connections on a single server (1999 paper by Dan Kegel). Traditional thread-per-connection model fails at scale (memory: 1MB stack × 10K = 10GB). Solution: event-driven I/O (NGINX event loop, Node.js, Netty). Load balancing distributes connections across multiple NGINX/Netty instances. Modern C100K/C1M problems solved with kernel bypass (DPDK), SO_REUSEPORT, eBPF-based load balancing (Cilium).

Q14. How does Nginx implement consistent hashing?

nginx upstream module: "hash \$request_uri consistent;" — uses consistent hashing based on request URI. Common for caching: all requests for same URL go to same upstream cache server, maximizing cache hit rate. "hash \$remote_addr;" for IP-based sticky sessions with consistent hashing (better than ip_hash when servers change). Ketama algorithm (same as Memcached) used for ring implementation.

Q15. What is the least response time algorithm?

Routes to server with combination of lowest active connections AND lowest recent average response time. More sophisticated than least connections alone — a slow server might have few connections but is still slow. Haproxy: "balance leastconn" + response time tracking. AWS ALB: least outstanding requests algorithm in routing. Overhead: must track response times per server. Best for APIs with high variance in response time.

Q16. How do you prevent a load balancer from being a single point of failure?

Active-active LB pair: both LBs active, DNS round-robins or ECMP routing between them. Active-passive: one primary, one standby with virtual IP (VRRP/HSRP). Floating IP moves to standby on failure. Cloud LBs (AWS ALB/NLB) are inherently redundant (multi-AZ). For DNS-level: Route53 health checks with failover routing. Anycast IP routes to nearest healthy LB globally (used by CDNs).

Q17. What is consistent hashing used for in Redis Cluster?

Redis Cluster uses hash slots (16384 total). $\text{Key} \rightarrow \text{CRC16}(\text{key}) \% 16384 = \text{slot}$. Cluster maps slot ranges to nodes. Adding/removing nodes: specific slot ranges migrate between nodes. Not pure consistent hashing (uses slots), but similar principle of minimizing remapping. Hash tags: `{user}.sessions` and `{user}.data` map to same slot (same node) by hashing only the part in braces — enables multi-key operations.

Q18. What is the thundering herd problem in caching and how does consistent hashing help?

Thundering herd: when a cache node fails, all keys that mapped to it miss simultaneously, flooding the DB with requests. With consistent hashing: only keys that mapped to the failed node move to its successor (K/N keys, not all). Virtual nodes spread the load: $150 \text{ vnodes per node} \times 4 \text{ remaining nodes} = \text{failed node's load}$ spread evenly. Additional mitigation: cache stampede prevention with mutex/probabilistic early expiry.

Q19. What is the role of a health check grace period?

Grace period: time after instance launch during which health checks are not evaluated. Prevents terminating instances that are still starting up (JVM init, DB connection pool warm-up, cache pre-population). AWS ASG: `HealthCheckGracePeriod` (default: 300s). Should match your application startup time + buffer. Too short: healthy instances terminated. Too long: failed instances left in pool too long. Measure actual startup time and set accordingly.

Q20. How does load balancing work in service mesh (Istio)?

Istio Envoy sidecar intercepts all service-to-service traffic. Performs L7 load balancing with algorithms: `ROUND_ROBIN`, `RANDOM`, `LEAST_CONN`, `PASSTHROUGH`. Client-side load balancing (no central LB). mTLS between services (mutual authentication). Supports: circuit breaking, retries, timeouts, fault injection, traffic splitting (canary). `DestinationRule` CRD configures load balancing policy per service.

Q21. What is the two-choice (power of two choices) algorithm?

Instead of picking one random server, pick two random servers and route to the one with fewer connections. Provably reduces maximum load from $O(\log N / \log \log N)$ to $O(\log \log N)$ with N servers — dramatic improvement. Used in: haproxy "balance random" with 2 choices, some CDNs. Trade-off: slight overhead of checking two servers. NGINX Plus supports this. Better than random, simpler than full least-connections tracking.

Q22. How does the key lookup work in a consistent hash ring?

$\text{hash}(\text{key})$ produces a long value mapped to ring position $0-2^{32}$. `TreeMap` (Java) or sorted array stores node positions. Lookup: `ceilingEntry(hash(key))` finds nearest clockwise node — $O(\log N)$ binary search on sorted structure. If $\text{hash}(\text{key}) > \text{max node position}$: wrap around to first node (`ring.firstEntry()`). Total lookup: $O(\log N)$ per request. Insertion of node: $O(V \log N)$ where V is vnodes count.

Q23. What is sticky sessions and when is it a problem?

Sticky sessions (session affinity): route each user always to the same server. Implemented via IP hash or LB-injected cookie (AWSALB cookie). Problem: if that server fails, user loses session. Hot spots: if one user generates 1000× traffic, their server overloads. Prevents true horizontal scaling. Best practice: eliminate stickiness by externalizing session to Redis. Only use stickiness as a last resort for legacy stateful applications.

Q24. How do microservices discover each other for load balancing?

Server-side discovery: services call API gateway → gateway queries service registry (Consul, Eureka) → gateway load-balances. Client-side discovery: service queries registry directly → client selects instance (Spring Cloud LoadBalancer with Eureka). DNS-based: service registers multiple A records → DNS round-robin (simple, no registry). Kubernetes: `ClusterIP` Service with kube-proxy performs load balancing via iptables/IPVS rules.

Q25. What is IPVS and how does it relate to load balancing?

IP Virtual Server: Linux kernel-level L4 load balancing built into netfilter. Kubernetes kube-proxy can use IPVS mode instead of iptables for better performance at scale (10K+ services). Algorithms: rr, wrr, lc, wlc, lbic (locality-based least connections — good for cache servers). IPVS uses hash tables → O(1) rule lookup vs iptables O(N) linear scan. Critical for large Kubernetes clusters.

Q26. What is the difference between DNS load balancing and hardware LB?

DNS: multiple A records for same hostname, TTL controls refresh (short TTL = faster failover but more DNS queries). Simple, no infrastructure, but: no session affinity, slow failover (TTL), client caching bypasses rotation, no health checks natively. Hardware LB (F5, Citrix): dedicated appliance with full L4/L7 features, SSL offload, WAF, excellent performance. Expensive, complex. Cloud LBs (ALB) are software-defined with hardware-like features.

Q27. How do you test load balancer configuration?

Health check: verify /health returns 200 before adding to pool, 503 triggers removal. Traffic distribution: send 100 requests, verify even distribution across instances. Failover: kill one instance, verify traffic reroutes within health check interval (10-30s). Connection draining: deploy new version, verify in-flight requests complete before old instance removed. Session persistence: IP hash sends same client to same server. Tools: wrk, hey, k6 for load generation; Wireshark for traffic inspection.

Q28. What are the limitations of consistent hashing?

Not perfectly uniform even with vnodes (statistical distribution). Hotspots possible if certain keys receive disproportionate traffic (not a distribution problem but an access pattern problem). Requires coordination to track ring topology (all nodes must agree). Hash function choice matters: MD5 provides good distribution; poor hash → clustered nodes. Very large vnode counts (>1000) increase memory and coordination overhead.

Q29. What is jump consistent hash?

A fast consistent hash algorithm by Google (2014) that maps a key to a bucket using only integer arithmetic — no ring data structure. O(ln N) time, O(1) space. Only supports adding servers at the end (no arbitrary removal). When bucket count changes, only 1/N keys remap. Ideal for: simple sharding where servers are added in order (no arbitrary removal). Implementation: 5-line function. Maglev hash (Google) extends this for load balancers.

Q30. How does the anycast IP technique relate to load balancing?

Anycast: same IP address announced from multiple geographic locations via BGP routing. Client packet routed to nearest anycast node by BGP "shortest path" metric. Cloudflare, AWS Global Accelerator use anycast. Benefits: automatic geo load balancing, DDoS absorption (attack traffic spread across all nodes globally), no DNS TTL issues (routing changes instantly at BGP level). Used for DNS servers (8.8.8.8), CDN edge nodes, NTP.

Q31. What happens during a hash ring rebalancing?

When a node is added: it takes over token ranges from its successor(s). Data for those ranges must be streamed from old node to new node (data migration). This is I/O intensive and can affect cluster performance. Cassandra uses streaming to move SSTables. Redis Cluster uses MIGRATE command for key migration. Best practice: add nodes gradually, monitor cluster load, use throttled migration to avoid overwhelming source nodes.

Q32. What is the NGINX upstream configuration for consistent hashing?

upstream backend { hash \$request_uri consistent; server 10.0.0.1:8080; server 10.0.0.2:8080; server 10.0.0.3:8080; } — routes same URI to same backend, maximizing cache hit rate. For health checks:

health_check interval=5s fails=2 passes=2 uri=/health; (NGINX Plus only). Open-source NGINX: passive health checks via proxy_next_upstream error timeout http_500. Timeouts: proxy_connect_timeout 5s; proxy_read_timeout 60s;

Q33. How do you implement blue-green deployment with a load balancer?

Maintain two identical target groups: Blue (current production) and Green (new version). LB routes 100% traffic to Blue. Deploy new version to Green, run tests. Switch LB to route 100% to Green. Monitor, keep Blue for quick rollback. After confidence: decommission Blue. AWS ALB: weighted target groups (blue=100, green=0 → blue=0, green=100). Zero downtime: LB drains Blue connections during switch. Connection draining timeout must match request duration.

Q34. What is the load balancer's role in rate limiting?

LB can enforce rate limits before requests reach application servers. AWS API Gateway: method-level throttling (per-stage, per-key). NGINX: limit_req_zone (token bucket per IP/key). Haproxy: stick-table based rate limiting. Benefits: protects all backend services uniformly, no per-service implementation needed. Limitation: LB-level rate limiting doesn't know business context (user tier, subscription). Combine with application-level for full control.

Q35. How does consistent hashing enable cache clustering?

Cache cluster (Memcached, Redis): each cache server holds a partition of the key space. Client uses consistent hashing to map keys to cache servers. When a server is added/removed, only K/N keys miss in cache — others stay warm. Without consistent hashing: entire cache invalidated on topology change (thundering herd). twemproxy (Nutcracker) and mcrouter implement consistent hashing transparently between application and Memcached cluster.